Lab *02 – Versioning Your Work*

Author: Mateus Guzman Fernandez
Course Section: 001
Date: 10/12/2021

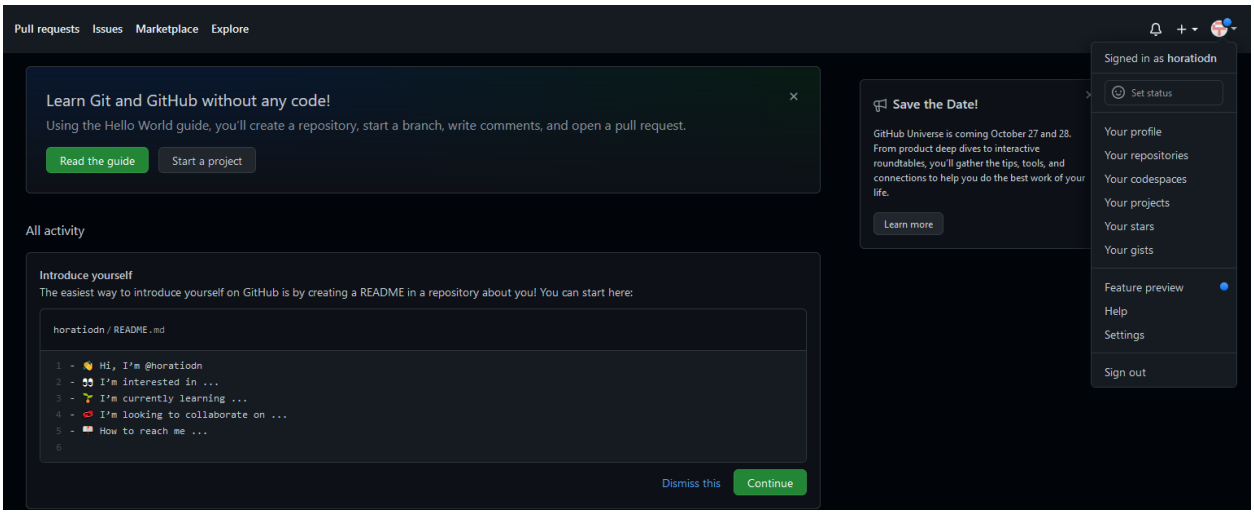
INTRODUCTION

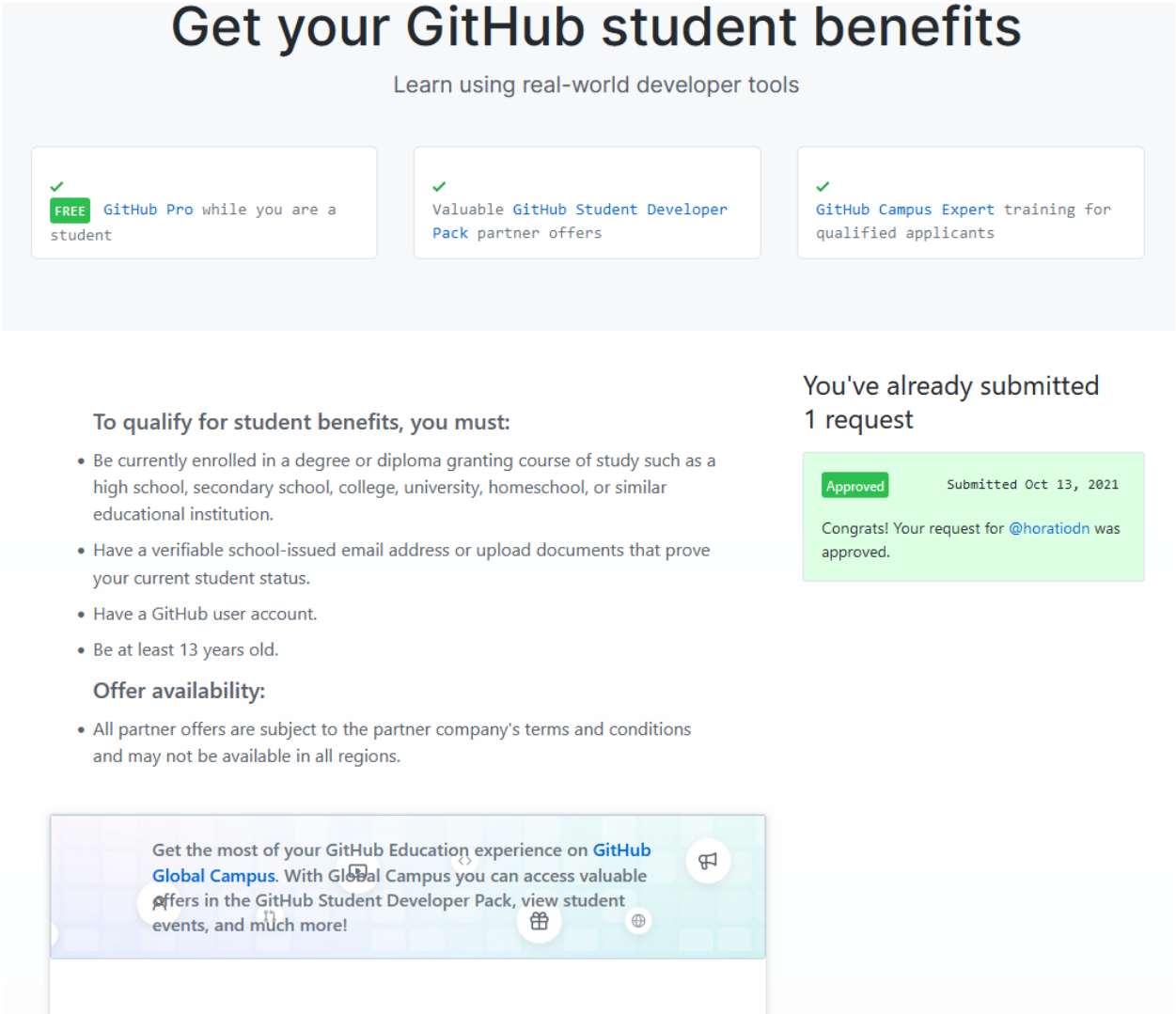
The objective of this lab is to provide a basic introduction to GitHub, an online, open-source version control system and provide useful knowledge on the set-up and purpose of the service. In this lab, we are going to provide a step-by-step guide to set up an account and give you a good foundation to build from as you begin your programming work. GitHub is a hugely useful service that is a cornerstone not just for software developers and programmers, but for many other fields and will surely be a great asset to have a good understanding of. With that being said, let's begin.

PROCESS

Breakpoint 1: GitHub

Down below, I have a snip of my GitHub account I made a few minutes ago. As you can see in the top right, my account is named "horatiodn" and my homepage is shown here. Getting student benefits is different and requires another process to get them. You must have a valid school email address and give them a reason to grant you student benefits to your account. You need to go to https://education.github.com/students, click the blue button and apply.





Breakpoint 2: What does Git do?

Here are some definitions of key Git operations:

Push: This is used to send committed files from the local repository to the remote repository. This will allow anyone with access to the remote repository the ability to see newly added files and changes.

Pull: This command takes files from the remote repository directly to the working directory. It's like using git merge and git fetch at once.

Fork: This creates a rough copy of a repository, which allows you to test and debug without affecting the original repository.

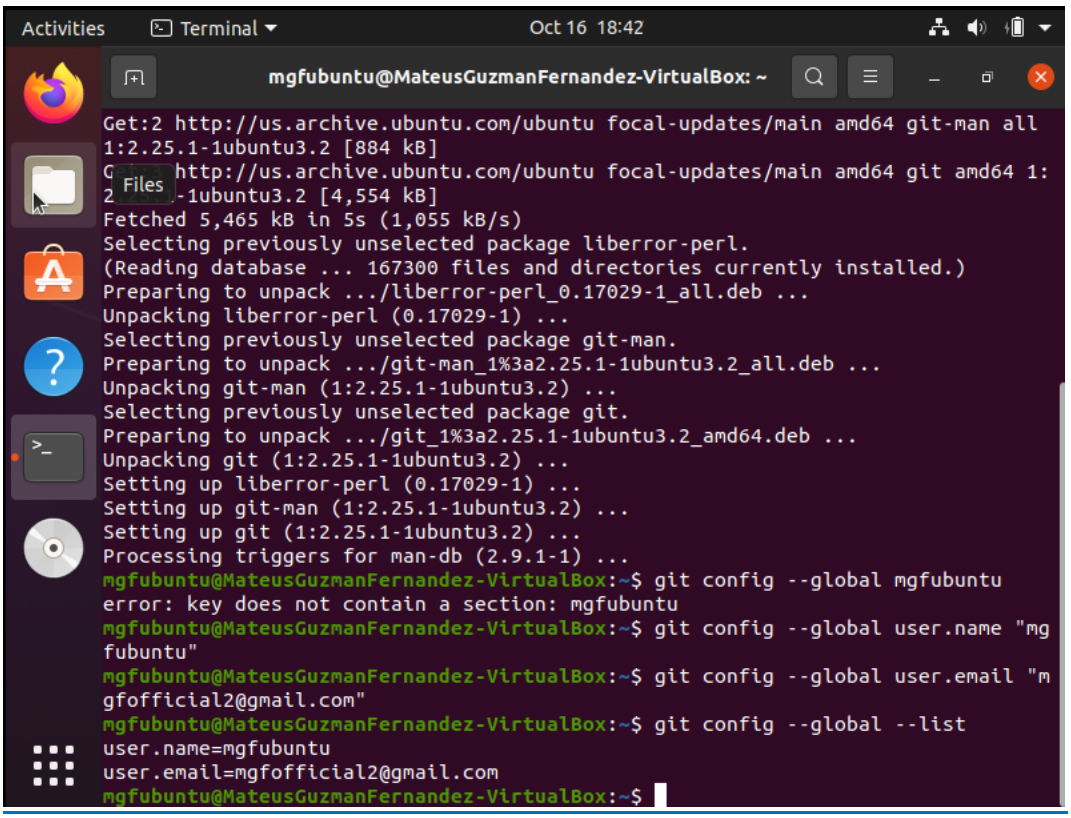
Commit: This sends files from the staging area to the local repository.

Clone: This creates a copy of a repository or a branch within that repository in another directory.

Branch: This lets you create, rename, delete, and list branches.

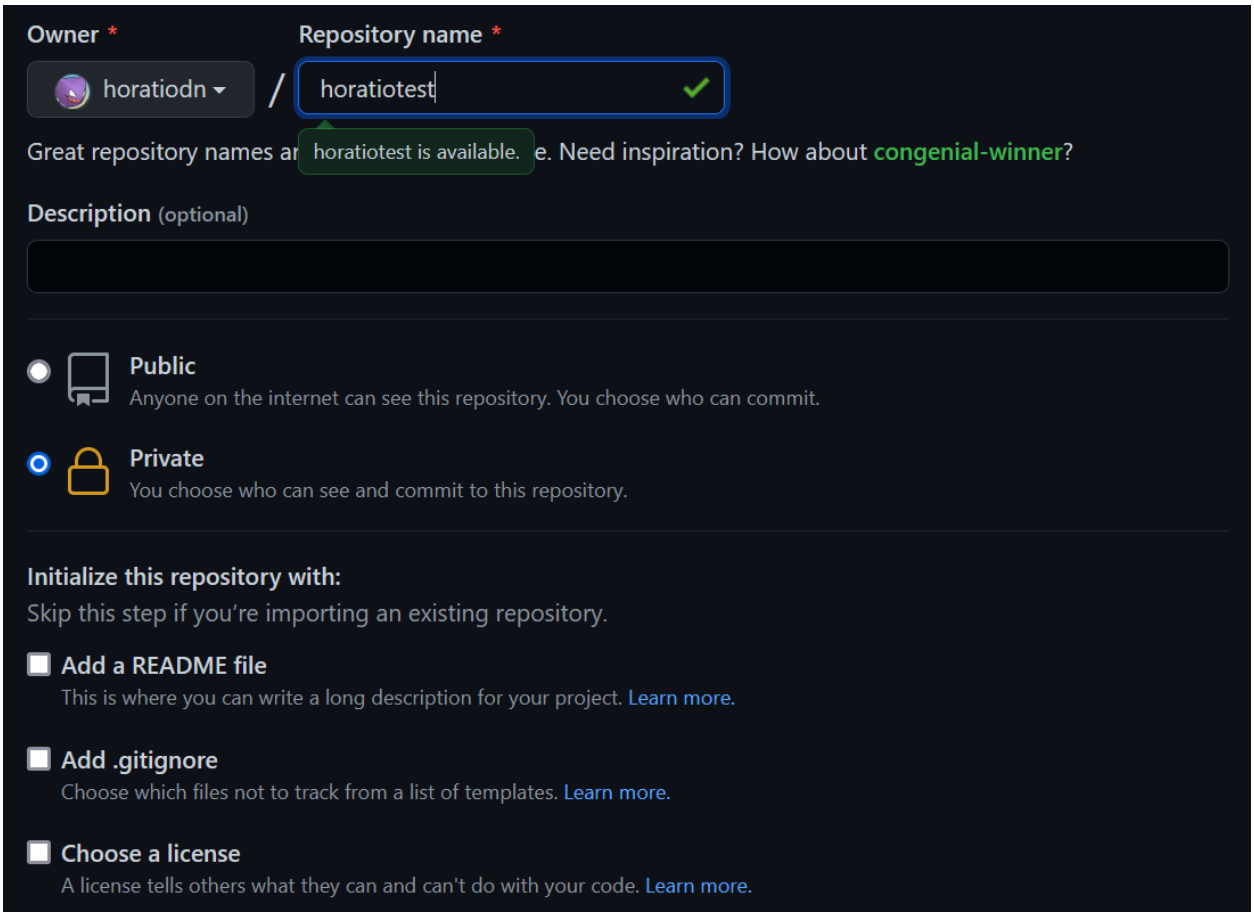
Breakpoint 3: Setting Up Git

Below, I have installed and set up Git on my Ubuntu virtual machine. I used the four commands below to set up the username, email, and check my configuration for the account. The command to set up a username is **git config –global user.name "username"**; to set up an email, type **git config --global user.email "email"**, and to check your configuration, type **git config –global –list**.



Breakpoint 4: Setting Up a GitHub Repository

This is what my new repository**, "horatiotest",** looks like right before I clicked the option to create it. I kept the default settings and changed from public to private.



Breakpoint 5: Committing your First Git Repo

This series of commands serves to set up our local repository (I'll refer to these as repos from now on).

First, I used **pwd** and **ls** to verify which directory I was in **(pwd)** and what the contents of my primary working directory was **(ls).** Next I added and deleted a couple of directories with **mkdir** and **rmdir**, before ultimately making the **"horatiotest"** directory with **mkdir** to match the name of the name of the repo I made on GitHub's website. After that, I used cd to hop into the new directory **"horatiotest"** and used redirection and echo to create a README.md file. I, then, checked to see if the file was there with ls and used **cat** to check the contents of the file, and sure enough, my message was there. My local repo was made!

Next, I used **git init** to initialize that local repo, and added it to the staging area with the **git add "filename"** command. Next, I committed the README.md file to the main branch of the repo with the **git commit -m "name"** command. I also converted the main branch to main from master using the **git branch -M main** command, which automatically worked for me*.

*If this does not work for you, try using **git branch -m master main** to do it manually.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ pwd
/home/mgfubuntu
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  temp  Templates  Videos
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ cd Desktop
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ ls
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ mkdir horatio
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ rmdir horatio
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ mkdir horatio
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ ls
horatio
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ rmdir horatio
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ mkdir horatiotest
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ cd horatiotest
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ echo "howdy and gig em" >> README.md
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ ls
README.md
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ cat README.md
howdy and gig em
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git init
Initialized empty Git repository in /home/mgfubuntu/Desktop/horatiotest/.git/
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git add README.md
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git commit -m "first commit"
[master (root-commit) 7f3a275] first commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git branch -M main
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$
```

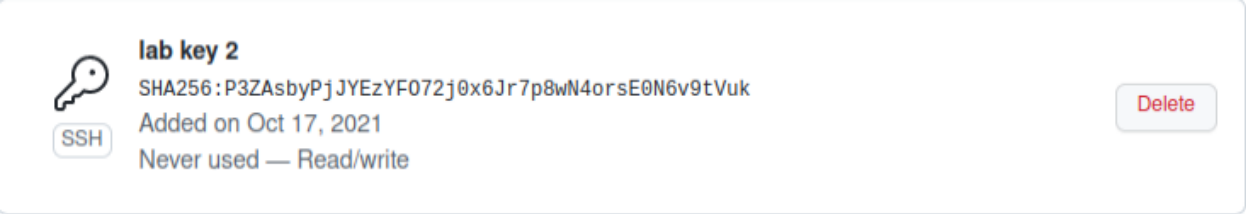
Breakpoint 6: Create a Secure Connection

I created my SSH key with the command "**ssh-keygen -t ed25519 -C "email",** after which I was prompted to choose what file to save it in. I just hit enter to save it in the default option **(/home/mgfubuntu/.ssh).** I also had to overwrite since I already had a file of the same name from a previous failed attempt where I forgot to capture a screenshot of my commands. After that, my (3rd) new SSH key was saved!

After that, I went changed my directory to ~/.ssh using cd and used ls -al to see all the files in it. I saw the SSH key file, **id_ed25519.pub**, and I used **cat** to check its contents. Sure enough, there was my key. I promptly added it to my GitHub account, and that's where we stand.

I used the SSH connection method because I had done it before and I was familiar with the commands.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/.ssh$ ssh-keygen -t ed25519 -C "mgfofficial2@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/mgfubuntu/.ssh/id_ed25519):
/home/mgfubuntu/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/mgfubuntu/.ssh/id_ed25519
Your public key has been saved in /home/mgfubuntu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:P3ZAsbyPjJYEzYFO72j0x6Jr7p8wN4orsE0N6v9tVuk mgfofficial2@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|        .. .      |
|       oo o o     |
|      o..o +      |
|   .   o... .     |
| . o . +S.+       |
|o . . o.+Bo+      |
|.=   .o.Oo* o     |
|..o  .+O E o      |
|  .o+*B+o         |
+----[SHA256]-----+
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/.ssh$ cd ~/.ssh
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/.ssh$ ls -al
total 16
drwx------  2 mgfubuntu mgfubuntu 4096 Oct 16 23:54 .
drwxr-xr-x 17 mgfubuntu mgfubuntu 4096 Oct 17 18:42 ..
-rw-------  1 mgfubuntu mgfubuntu  464 Oct 17 18:56 id_ed25519
-rw-r--r--  1 mgfubuntu mgfubuntu  104 Oct 17 18:56 id_ed25519.pub
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/.ssh$ cat id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKiPfgELwRgC762WCOvbBH4A9ILT8OVDm81NvDo7kwa3 mgfofficial2@gmail.com
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/.ssh$
```



Breakpoint 7: Sync Your Repos

I first needed to move into my repository folder I made on my Desktop to do this part. I used **cd** a few times to get there. To add my local repo to the remote repo on GitHub, I used **"git remote add origin "repo link"**. Now, my repo is prepared, but nothing has changed in terms of the data. I could have also used "**git remote -v**" to see if I had already created this repo, but I was certain that it was new.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/.ssh$ cd ..
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ cd Desktop
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ ls -al
total 12
drwxr-xr-x  3 mgfubuntu mgfubuntu 4096 Oct 16 22:23 .
drwxr-xr-x 18 mgfubuntu mgfubuntu 4096 Oct 17 19:09 ..
drwxrwxr-x  3 mgfubuntu mgfubuntu 4096 Oct 16 22:37 horatiotest
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop$ cd horatiotest
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git remote add origin git@github.com:horatiodn/horatiotest
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$
```

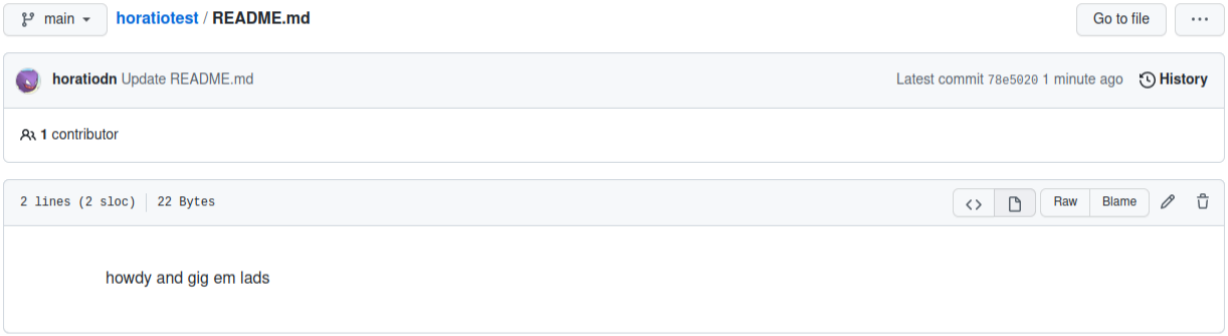
Breakpoint 8: Pushing Changes on Your Local Repo to Your Remote Repo

The command "**git push -u origin main**" has your repos track any changes made when you next go to synchronize them. Your repos will now also reject any changes if they create conflicts.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git push -u origin main
The authenticity of host 'github.com (140.82.112.4)' can't be established.
RSA key fingerprint is SHA256:nThbg6kXUpJWGl7E1IGOCspRomTxdCARLviKw6E5SY8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'github.com,140.82.112.4' (RSA) to the list of known hosts.
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 229 bytes | 229.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To github.com:horatiodn/horatiotest
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$
```

Breakpoint 9: Make an Edit to Your Remote Repo

I went onto GitHub and added "lads" to my original README.md's text, which just read "howdy and gig em". The change was committed to the file using a green button on the GUI.



Breakpoint 10: Pulling Changes from Your Remote Repo to Your Local Repo

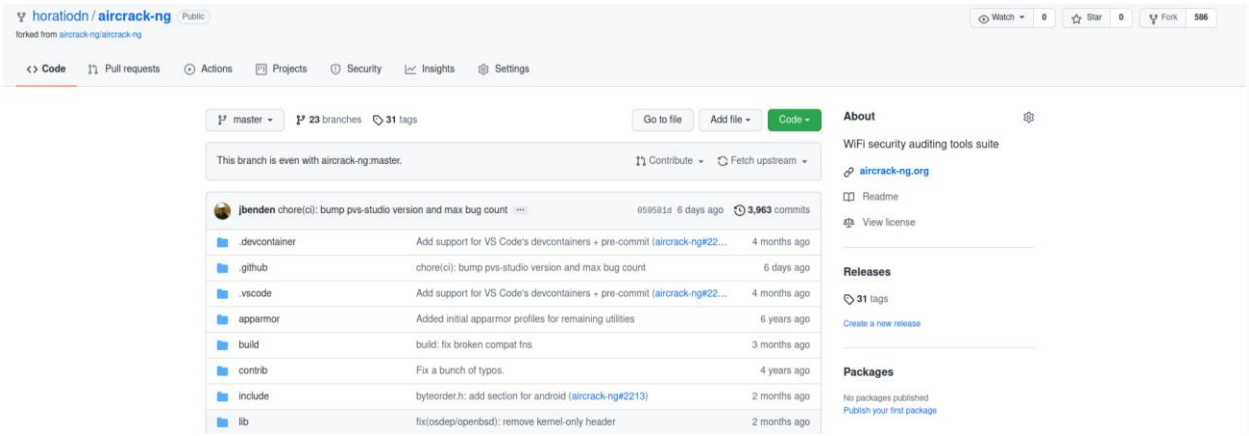
I used the command "**git pull origin main**" to pull the changes to the README.md file from the remote repo (GitHub) to my local repo on the Linux machine. The command read that 1 change was made to the README.md file – an insertion, denoted by a + sign.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git pull origin main
Warning: Permanently added the RSA host key for IP address '140.82.114.3' to the list of known hosts.
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), 1.17 KiB | 1.17 MiB/s, done.
From github.com:horatiodn/horatiotest
 * branch            main       -> FETCH_HEAD
   7f3a275..78e5020  main       -> origin/main
Updating 7f3a275..78e5020
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$
```

Breakpoint 11: Forking and Cloning a GitHub Repo

I chose to fork aircrack-ng because we worked with this suite several times last year in our Cyber class and it instantly came to me when I read the directions. I had never directly worked with GitHub, but we did use this on Kali Linux when we did labs last year, so yeah, shoutout Mr. Bobo's Cyber class at Miller.

I went and cloned the aircrack-ng repo using **git clone "link"** to my own local repo, and now I can work on the files of the suite locally, without affecting the main repo.




```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git clone git@github.com:horatiodn/aircrack-ng.git
Cloning into 'aircrack-ng'...
Warning: Permanently added the RSA host key for IP address '140.82.113.3' to the list of known hosts.
remote: Enumerating objects: 28027, done.
remote: Counting objects: 100% (570/570), done.
remote: Compressing objects: 100% (295/295), done.
remote: Total 28027 (delta 304), reused 487 (delta 259), pack-reused 27457
Receiving objects: 100% (28027/28027), 15.41 MiB | 9.11 MiB/s, done.
Resolving deltas: 100% (19780/19780), done.
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$
```

Breakpoint 12: Branching a Repo

Branching repos make organization very easy in terms of managing the different commits in your repo. A branch is a pointer to a certain commit, almost like a snapshot. Branches aren't stored in folders, nor are they folders themselves; they are all stored in a flat manner, making them equal.

I am going to create a branch using the command **git checkout -b "branch name"**.

Checkout is a special command here because it has a load of features. Firstly, it finds the commit the branch inputted by the user is referring (pointing) to and updates all the files in the working tree to match the files in the commit. Next, it switches HEAD to the branch inputted, so all commits will now have pointers to the last commit made in the branch you checked out.

The flag "**-b**" is what will create a branch with the name you desire.

Next, I'm going to push this branch to the remote repo. Right now, it is in my working directory, but I want to send it to the remote repo at GitHub, so I will run the command **git push –set-upstream origin "branch-name"**

This command tells the remote server that a branch has been created client-side, and the server will recreate that branch there. The flag **–set-upstream** formalizes the relationship between the branch on the server and the local branch.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git checkout -b horatio-branch
Switched to a new branch 'horatio-branch'
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$ git push --set-upstream origin horatio-branch
Warning: Permanently added the RSA host key for IP address '140.82.112.3' to the list of known hosts.
Total 0 (delta 0), reused 0 (delta 0)
remote:
remote: Create a pull request for 'horatio-branch' on GitHub by visiting:
remote:      https://github.com/horatiodn/horatiotest/pull/new/horatio-branch
remote:
To github.com:horatiodn/horatiotest
 * [new branch]      horatio-branch -> horatio-branch
Branch 'horatio-branch' set up to track remote branch 'horatio-branch' from 'origin'.
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~/Desktop/horatiotest$
```

CONCLUSION

The goals of the tools we used in this lab are to give a basic understanding of how git and GitHub work and to introduce us to useful commands and concepts relating to them. They are highly relevant to our degrees and future careers as GitHub and git are some of the most widely used version control platforms and learning how they work and what commands make them run are imperative to many careers in Cyber, especially software engineers. These tools are highly effective as they form the basis of the knowledge needed to operate important platforms for use throughout and exceeding Cyber fields. As for other aspects, taking a closer look at branching would be interesting to me. I think the lab is highly relevant and only became a bit more challenging with the back and forth in GitHub and the bash, the higher number of commands we had to use, and the

last, open-ended section to round it off. Overall, I'm really pleased with the lab, and I can't wait to see what else we have down the road in the class!

References

Internet Resources

1. https://education.github.com/students (GitHub for Students, GitHub)
2. An Introduction to GitHub (The GitHub Training Team, 29 July 2020)
3. 1.1 Introduction: Git and GitHub for Poets (The Coding Train, 19 April 2016)
4. Learn the Basics of Git in Under 10 Minutes (freeCodeCamp, 5 Jan 2019).
5. GitHub SSH Pages (About SSH, GitHub)
6. Git Create Branch (Zilberman, Datree.io, 2021)
7. GitHub to replace with 'master' with 'main' starting next month (Cimpanu, ZDNet, 2020)

Collaboration

I did not collaborate with anyone on this lab.